TELUGU ICON

Digital & Influencer Awards 2026

OPERATIONS RUNBOOK

Deployment | Scaling | Load Balancing | API Reference

API keys & endpoint reference

Environment variables to change

Capacity for 10k / 30k / 50k / 100k concurrent users

Step-by-step production deployment

16 November 2026 | Shilpakala Vedika, Hyderabad

Organised by Pracharam Media

Generated 2026-10-12 - capacity figures measured on this codebase

Internal document. Contains no production secrets.

0. Contents

How to read this document

Sections 1-3 are setup and reference material. Section 4 contains the capacity numbers measured on this codebase - they are not estimates. Sections 5-7 are what you actually need on launch day.

1	Project Overview - Stack, structure, what talks to what
2	Environment Variables - Every variable, default, and when to change it
3	API Reference - Keys, auth, every endpoint, rate limits
4	Measured Capacity - Real load-test numbers for this code
5	Scaling to 10k / 30k / 50k / 100k - Instances, DB, load balancer sizing
6	Load Balancer Setup - nginx and managed-LB configuration
7	Production Deployment - Step-by-step, local to live
8	Pre-Launch Checklist - The things that break on the night

1. Project Overview

1.1 What this is

A public voting and event-ticketing platform for the Telugu Icon Digital & Influencer Awards 2026. Visitors submit self-nominations, OTP-verify their mobile number, and cast one vote per category. Admins review nominations, promote them to nominee pages with shareable campaign kits, and manage sponsors, tickets and site settings.

1.2 Stack

LAYER	TECHNOLOGY
Frontend	React 18, Vite, Tailwind CSS, React Router, Framer Motion
Backend	Node.js, Express 4, JWT auth, bcryptjs, multer (uploads)
Database	MySQL / MariaDB via mysql2, InnoDB, utf8mb4_unicode_ci
Auth	Admin = JWT (jsonwebtoken) + bcrypt. Voter = OTP (MSG91, dev fallback)
QR	qrcode library, generated locally in-browser. PDF via jsPDF

1.3 Repository layout

```
aibuild/
  backend/                Express API
    config/database.js    mysql2 connection pool
    database/schema.sql   ALL 23 tables + seed data (idempotent)
    routes/               19 route modules
    scripts/              db setup, seeding, audits, load test
    uploads/              nomination photos
    .env                  environment variables (never commit)
  frontend/               React app
    src/pages/            public pages
    src/pages/admin/      admin dashboard (13 tabs)
    src/components/       Navbar, Footer, QrBlock, OtpVote, ...
    public/               static assets, logo, OG image
    .env                  frontend variables
```

1.4 Request flow

```
Browser
  |  GET /                    Vercel (CDN) serves the React app
  |
  |  fetch /api/*             -> nginx / Railway load balancer
  v
Express (1..N instances)
  |  jwt auth        (admin routes)
  |  rate limit      (in-memory, per process)
  v
mysql2 pool (DB_POOL_SIZE per instance)
  |
  v
MySQL / MariaDB  -  telugu_icon_awards (23 tables)
```

1.5 Ports

PORT	SERVICE	NOTES
5173	Vite dev server (npm run dev)	Frontend only, local dev
3001	Express API	Backend, local dev
3307	MariaDB (XAMPP)	Note: your XAMPP is on 3307, not 3306
80/443	nginx	Production reverse proxy

> **Check your XAMPP port**
> C:\xampp\mysql\bin\my.ini sets port=3307 on this machine. If you change it, DB_PORT in backend/.env must match. The server auto-creates the database and applies schema.sql on boot if it is missing.

2. Environment Variables

Two files. backend/.env runs the API. frontend/.env is baked into the browser bundle at build time - changing it requires a rebuild, not just a restart.

2.1 backend/.env - core

VARIABLE	DEFAULT	WHAT IT DOES
PORT	3001	API port. Set by the platform in production.
NODE_ENV	production	Set by the platform. Do not hardcode.
DB_HOST	127.0.0.1	Local only. In production use the managed DB hostname.
DB_PORT	3307	MATCHES YOUR XAMPP. Use 3306 for standard MySQL.
DB_USER	root	XAMPP default is root with an empty password.
DB_PASSWORD	(empty)	NEVER COMMIT. Use a real password in production.
DB_NAME	telugu_icon_awards	Created automatically if absent.
DB_TIMEZONE	Z	Use +05:30 to write DATETIME in IST.
DB_POOL_SIZE	10	Per instance. See section 5.4 before raising.
DB_QUEUE_LIMIT	50000	Pending query queue. 50000 is fine.
JWT_SECRET	change-me	REQUIRED IN PRODUCTION. Long random string.
JWT_EXPIRES_IN	24h	Admin session length.
FRONTEND_URL	*	CORS origin. Set to your domain to lock it down.
PUBLIC_URL	(unset)	Keep-alive ping URL used in production.

2.2 backend/.env - traffic

VARIABLE	DEFAULT	WHAT IT DOES
TRUST_PROXY	false	MUST BE true behind nginx/Cloudflare/Railway. See callout.
CLUSTER	0	1 enables Node cluster mode.
WORKERS	1	Worker count when CLUSTER=1. Match your CPU count.
VOTE_RATE_MAX	20	Votes per IP per window. Flood guard only.
VOTE_RATE_WINDOW_MINUTES	60	Window for the above.

TRUST_PROXY - the single most important variable
Without it, req.ip is the load balancer address, so every visitor looks like one shared IP. The per-IP vote limit would then be exhausted by 20 requests total, and the (ip_address, user_agent) duplicate guard would let the FIRST person to vote lock out the entire audience. Set TRUST_PROXY=true in any deployed environment.

2.3 backend/.env - SMS (OTP)

VARIABLE	DEFAULT	WHAT IT DOES
MSG91_AUTH_KEY	set locally	From control.msg91.com. Without it OTPs are simulated.
MSG91_TEMPLATE_ID	(empty)	DLT template id. Both needed for real SMS.
MSG91_OTP_EXPIRY	10	OTP validity in minutes.

When MSG91_TEMPLATE_ID is empty, sendOtp returns the code in the API response as dev_otp. That is how local voting works with no SMS cost. It MUST NOT be enabled in production - anyone could vote without owning the number. Set both variables, or reject unverified votes server-side.

2.4 frontend/.env

VARIABLE	DEFAULT	WHAT IT DOES
VITE_API_URL	(unset)	API base. Unset in dev = /api through the Vite proxy.
VITE_SITE_URL	(unset)	Canonical/OG/QR origin. Leave unset to derive from request.
VITE_PROXY_TARGET	http://localhost:3001	Dev only. Where Vite forwards /api.

VITE_SITE_URL should stay unset unless you need to force a different origin than the one being served. Setting it wrongly breaks every share link, QR code and sitemap URL at once.

2.5 Production .env checklist

MUST CHANGE	TO WHAT
JWT_SECRET	Long random string. Rotating it logs out every admin.
DB_PASSWORD	Non-empty, from the managed provider.
TRUST_PROXY	true
FRONTEND_URL	https://your-domain.com (not *)
CLUSTER / WORKERS	1 / (CPU count) on a multi-core box
MSG91_TEMPLATE_ID	Set, or voting is unverified

3. API Reference

Base URL in development: http://localhost:5173/api (proxied to :3001). In production: https://your-domain.com/api

3.1 Which key goes where

There are only two credentials, and neither is an API key:

CREDENTIAL	WHO	HOW IT IS USED
`JWT token`	Admin only	POST /api/auth/login returns it. Send as: Authorization: Bearer <token>. Stored in localStorage as admin_token. 24h expiry.
`OTP code`	Voters	Sent to the mobile number. POST /api/otp/verify exchanges it for a verification_token (30 min). No API key involved.

There is no third-party API key in the voting path. MSG91_AUTH_KEY is only for outbound SMS delivery.

3.2 Auth flow (admin)

```
POST /api/auth/login
  { "email": "admin@teluguiconawards.com", "password": "admin123" }
  -> 200 { "token": "eyJ...", "user": { "id":1, "role":"super_admin" } }

then every admin call:
  Authorization: Bearer eyJ...
```

3.3 Voter flow (OTP + vote)

```
1) POST /api/otp/send      { "mobile": "9876543210" }
   -> { "type":"success", "otp":"123456", "dev_otp":"123456" }
   (10s cooldown per number; 429 after that)

2) POST /api/otp/verify    { "mobile": "9876543210", "otp": "123456" }
   -> { "type":"success", "verification_token":"<64 hex>", "mobile":"919876543210" }

3) POST /api/votes         { "nominee_id":1, "voter_name":"...", "voter_email":"...",
                            "voter_phone":"9876543210",
                            "verification_token":"<from step 2>",
                            "device_fingerprint":"<optional>" }
   -> 201 { "message":"Your vote has been recorded...", "vote_count": 2 }

Duplicate votes in the same category -> 400:
   "This mobile number has already voted in this category."
   "This email address has already voted in this category."
   "You have already voted in this category from this device."
```

3.4 Optional pre-check (saves the OTP cost)

Call this BEFORE requesting an OTP. If it returns already_voted=true the number has already voted in that category and no SMS is needed - the UI skips straight to the "Vote Already Recorded" screen.

```
GET /api/votes/check?phone=9876543210&nominee_id=1
  -> { "already_voted": true, "nominee_name":"...",
       "category_name":"...", "voted_at":"2026-10-12T23:52:56.000Z" }
  -> { "already_voted": false }
```

It exposes only whether a vote exists, never the voter name, email or IP. It is advisory: POST /api/votes still enforces the limit on its own.

3.5 Public endpoints (no auth)

`GET`	`/api/health`	Service check - used by uptime checks
`GET`	`/api/categories`	All 50 categories (also ?jury, ?self-nomination, /:slug)
`GET`	`/api/nominees`	Approved nominees. ?category= accepts slug OR id
`GET`	`/api/nominees/featured`	Featured nominees, max 12
`GET`	`/api/nominees/:id`	Single nominee - accepts numeric id or slug
`GET`	`/api/settings`	Site settings key/value rows

GET	/api/sponsors	Active sponsors
GET	/api/gallery	Published gallery items
GET	/api/media	Press/press-room items
GET	/api/media/videos	Video hub items
GET	/api/past-events	Past editions
GET	/api/past-events/:slug	One edition + its awardees
GET	/api/awardees	Winners. ?event_id= &year= &category= &name=
GET	/api/tickets/availability	Public ticket tiers (never exposes inventory)
GET	/api/otp/mode	"otp" or "device"
GET	/api/votes/check	Pre-vote duplicate check (section 3.4)
POST	/api/otp/send	Request OTP
POST	/api/otp/resend	Resend OTP
POST	/api/otp/verify	Verify OTP, returns verification_token
POST	/api/votes	Cast a vote (section 3.3)
POST	/api/nominations	Submit a self-nomination (multipart, photo upload)
POST	/api/contact	Contact form
POST	/api/sponsor-leads	Become-a-partner enquiry
POST	/api/tickets/book	Book tickets (mock payment)
GET	/api/tickets/order/:qr_token	Digital ticket by token
POST	/api/tickets/verify-entry	Gate scanner endpoint
POST	/api/nomination-status/send	OTP for the nomination tracker
POST	/api/nomination-status/verify	Returns nomination_status token
POST	/api/nomination-status/lookup	Nominations + campaign kit for that mobile
POST	/api/analytics/track	Campaign analytics (deduplicated)

3.6 Admin endpoints (Authorization: Bearer <token>)

GET	/api/admin/stats	Dashboard counters + last 10 votes
GET	/api/admin/nominations	All nominations (?status=)
PUT	/api/admin/nominations/:id	Set status. "shortlisted" promotes a nominee
DEL	/api/admin/nominations/:id	Delete a nomination
GET	/api/admin/nominees	All nominees + campaign counters
POST	/api/admin/nominees	Create nominee (mints TIC26-##### code)
GET	/api/admin/nominees/:id/campaign	MY CAMPAIGN panel + 14-day series
PUT	/api/admin/nominees/:id	Update. Slug only changes if explicitly sent
DEL	/api/admin/nominees/:id	Delete nominee
GET	/api/admin/votes	Vote audit list
PUT	/api/admin/votes/:id/flag	Flag / unflag
DEL	/api/admin/votes/:id	Delete vote, decrements vote_count
GET	/api/admin/votes/export	CSV download
GET/POST/PUT/DEL	/api/admin/sponsors	Sponsor CRUD

`GET /api/admin/sponsors`	Sponsor CRUD
`GET/PUT /api/admin/settings`	Read / upsert site settings
`PUT /api/admin/event-phase`	Switch phase_1..phase_5
`GET /api/admin/messages`	Contact inbox
`GET /api/admin/tickets`	Ticket configs + orders + revenue stats
`PUT /api/admin/tickets/:id/check-in`	Gate check-in
`PUT /api/admin/tickets/config/:id`	Edit a ticket tier
`GET /api/admin/audit-logs`	Audit trail
`GET /api/admin/past-events`	Past events CRUD
`GET /api/admin/awardees`	Awardees CRUD
`GET /api/admin/gallery`	Gallery CMS CRUD
`GET /api/admin/media`	Media CMS CRUD
`GET /api/admin/sponsor-leads`	Partner enquiry inbox
`GET/POST/DEL /api/admin/sms-templates`	DLT templates + sender IDs
`POST /api/admin/upload`	Image upload (multipart field: image)

3.7 Rate limits and abuse controls

CONTROL	LIMIT	NOTES
`Vote, per IP`	20 / hour	VOTE_RATE_MAX. In-memory, PER PROCESS.
`OTP send, per mobile`	15 / 24h	Plus a hard 10s gap between sends.
`Nomination lookup, per IP`	20 / hour	In-memory, per process.
`Analytics track, per fingerprint`	30-1800s	Window varies by event type.
`One vote per category`	Hard limit	DB unique keys. The REAL limit - see below.

The authoritative duplicate protection is in the database, not in code:

```
UNIQUE KEY uq_votes_category_phone  (category_id, voter_phone)
UNIQUE KEY uq_votes_category_email  (category_id, voter_email)
UNIQUE KEY uq_votes_category_device (category_id, device_fingerprint)
```

These hold across every API instance. The in-memory IP limit is flood protection only - it multiplies by worker count under clustering, so it is never the limit you should rely on.

4. Measured Capacity

These are measured numbers from this exact codebase, not vendor figures. Reproduce with:

```
cd backend
node scripts/load-test.js read --concurrency=500 --duration=20
node scripts/load-test.js vote --concurrency=200 --duration=20
```

Measured on: 6-core / 12-thread CPU, 15 GB RAM, MariaDB 10.4.32, one Express process, XAMPP on the same box. The load generator shared that CPU, so production with a dedicated generator will do better.

4.1 Read path (GET)

CONCURRENT	THROUGHPUT	P50	P99	ERRORS
200	4052 req/s	46 ms	86 ms	0%
500	3994 req/s	120 ms	229 ms	0%
1000	3805 req/s	243 ms	533 ms	0%
2000	3520 req/s	526 ms	1264 ms	0%

Reads hold up well. Throughput barely falls (4052 to 3520) - only latency grows. A single process absorbs 1000-2000 concurrent visitors with zero errors.

4.2 Write path (voting)

CONCURRENT	THROUGHPUT	P50	P99	ERRORS
200	1193 votes/s	158 ms	271 ms	0.01%
1000	822 votes/s	1253 ms	1534 ms	0.07%

~1000 concurrent voters is the knee. Past that, latency passes 1.5s, users start abandoning and double-tapping, which creates MORE load.

4.3 Clustering did not help writes

CONFIG	READ	VOTE
1 process	4052 req/s	1193 votes/s
6 workers	3994 req/s	1164 votes/s

Six workers gave no gain. The bottleneck is not Node and not the connection pool - it is MySQL row locking. Confirmed by the lock counters during the 1000-concurrent vote run:

```
Innodb_row_lock_waits        10653      <- 0.66 lock waits per vote
Innodb_row_lock_time_ms      10819
Innodb_row_lock_time_max       218 ms
Max_used_connections            11      <- pool limit is 10: NOT connection-bound
Connection_errors_max_conn      0
Slow_queries                    0
```

Every vote runs UPDATE nominees SET vote_count = vote_count + 1 inside the transaction, so every vote for the same nominee fights for one row. Section 8 lists the fix.

4.4 What the numbers mean in plain terms

INSTANCES	BROWSING (concurrent)	VOTING (concurrent)
1	1,000 - 2,000	700 - 1,100
2	2,000 - 4,000	1,400 - 2,200
5	5,000 - 10,000	3,500 - 5,500
10	10,000 - 20,000	7,000 - 11,000
20+	20,000+	14,000+

5. Scaling to 10k / 30k / 50k / 100k

First, separate two things people conflate. Registered members are database rows and cost nothing. Simultaneous users are what need capacity.

5.1 Registered members: not a scaling problem

10k, 50k or 500k registered members are all just rows. MySQL handled 16,000 votes in 20 seconds without strain. Size storage, not servers:

5.2 Work out your real peak

Voting rate is the only number that matters. If N users vote within a window T seconds:

```
votes per second = N / T

 20,000 members over 24 hours   ->    0.2 votes/s   -> 1 instance
 20,000 members over  1 hour    ->    5.6 votes/s   -> 1 instance
 20,000 members over  5 minutes ->   67   votes/s   -> 1 instance
 20,000 members in  60 seconds  ->  333   votes/s   -> 1 instance
100,000 members in  60 seconds  -> 1667   votes/s   -> 3 instances
100,000 members in  30 seconds  -> 3333   votes/s   -> 5 instances
```

Voting is almost never your problem. Page loads are.

5.3 Instance count for simultaneous visitors

Each visitor fires roughly 4-6 API calls on first load (settings, categories, nominees, sponsors, gallery, analytics). At 4000 req/s per instance:

CONCURRENT	API CALLS	DRAIN TIME	BROWSING INST.	VOTING INST.
10,000	50,000 req	13 s	5-10	1
30,000	150,000 req	38 s	15-30	2-3
50,000	250,000 req	63 s	25-50	4-6
100,000	500,000 req	125 s	50-100	10-15

"Drain time" is how long a burst takes to clear at full throughput. The real defence is a CDN: if static assets and read-heavy GETs are cached at the edge, most of those 500,000 calls never reach your instances at all.

5.4 Database sizing

This is the part people skip and it is the part that actually bites at 50k+.

MySQL VARIABLE	NOW	SHOULD BE	WHY
innodb_buffer_pool_size	16 MB	512M - 2G	CRITICAL. XAMPP default is 16MB.
max_connections	151	instances x pool + 20%	Must exceed total pool size.
DB_POOL_SIZE	10	10-25	Per instance. See below.
innodb_flush_log_at_trx_commit	1	2	Group commit. Safer trade: up to 1s of vote loss on power cut.
table_open_cache	default	4000	Many tables + prepared statements.

On pool size: total connections = instances x DB_POOL_SIZE. For 20 instances at pool 15 that is 300 connections, so max_connections must be at least ~360. Exceeding it shows up as "Too many connections" and a dead site. Do not set DB_POOL_SIZE to 100 thinking it goes faster - the database will thrash. Between 10 and 25 is the useful range.

```
# C:\xampp\mysql\bin\my.ini  (restart MySQL from the XAMPP panel after editing)
[mysqld]
innodb_buffer_pool_size = 1G
max_connections            = 500
innodb_flush_log_at_trx_commit = 2
innodb_log_file_size       = 512M
table_open_cache           = 4000
slow_query_log             = 1
slow_query_log_file        = C:/xampp/mysql/logs/slow.log
long_query_time            = 1
```

5.5 Where to run each layer at scale

CONCURRENT	APP	DATABASE	NOTES
`0-10k concurrent`	1 instance, 2 vCPU / 4 GB	Managed MySQL or the same box	Enough.
`10k-30k`	5-15 instances behind nginx	Managed MySQL (RDS/PlanetScale)	CDN strongly advised.
`30k-50k`	25-50 instances, 2+ servers	Managed MySQL, read replica	CDN required.
`50k-100k`	50-100 instances, autoscale	Managed MySQL or sharded read	CDN + multi-region.

Do not put 100k users on one XAMPP box
A local XAMPP instance is fine for development and for a few hundred concurrent users. Beyond that the box becomes both the web tier and the database, they fight for the same CPU, and a single disk failure takes everything down. Move the database to a managed host first - that is the highest-value change.

6. Load Balancer Setup

Required as soon as you run more than one instance. It also fixes IP handling, which is why TRUST_PROXY matters immediately.

6.1 nginx (recommended - full control, near-zero cost)

```
upstream telugu_api {
    least_conn;                       # better than round-robin for long polls
    server 127.0.0.1:3001 max_fails=3 fail_timeout=30s;
    server 127.0.0.1:3002 max_fails=3 fail_timeout=30s;
    keepalive 64;
}

server {
    listen 80;
    server_name your-domain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name your-domain.com;

    ssl_certificate     /etc/letsencrypt/live/your-domain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/your-domain.com/privkey.pem;

    # Uploads can be up to 10MB (admin images).
    client_max_body_size 12M;

    location /api/ {
        proxy_pass         http://telugu_api;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        # REQUIRED: without this every visitor looks like one IP.
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_set_header   Connection        "";
        proxy_read_timeout 30s;
    }

    location /uploads/ {
        proxy_pass         http://telugu_api;
        proxy_set_header   Host $host;
        expires 7d;
    }

    # Static frontend, served by Vercel/CDN in most deployments.
}
```

Then set TRUST_PROXY=true in backend/.env so Express honours X-Forwarded-For.

6.2 Managed load balancer

If you would rather not maintain nginx, any of these work with no code changes:

OPTION	NOTES
`Railway / Render / Fly`	Built-in. Set TRUST_PROXY=true and redeploy.
`Cloudflare`	Free tier. Also caches static assets and terminates TLS.
`AWS ALB / GCP LB`	Health checks on /api/health. Target the Express port.
`Nginx Proxy Manager`	GUI on top of nginx if you want certs managed.

6.3 Health checks

```
GET /api/health  ->  200 { "status":"ok", "db":"3307" }

# nginx: every 10s, 3 failures before ejecting an instance
location = /api/health {
    proxy_pass http://telugu_api;
    access_log off;
}
```

7. Production Deployment

7.1 Prerequisite checklist

- Managed MySQL provisioned, schema applied, innodb_buffer_pool_size raised
- Domain pointed at the load balancer, TLS certificate installed
- backend/.env complete - especially JWT_SECRET, DB_PASSWORD, TRUST_PROXY, MSG91_TEMPLATE_ID
- MSG91 DLT template approved (real OTP delivery, or voting is unverified)
- Admin password changed from the default admin123
- One instance live and healthy behind the load balancer

7.2 Deploy the backend

```
# on the server
cd /srv/aibuild/backend
npm ci --omit=dev
cp .env.example .env       # then edit it

# schema is applied automatically on boot, or run it explicitly:
npm run db:setup

# systemd unit: /etc/systemd/system/tia-api.service
[Unit]
Description=Telugu Icon Awards API
After=network.target mysql.service

[Service]
Type=simple
User=www-data
WorkingDirectory=/srv/aibuild/backend
Environment=NODE_ENV=production
Environment=CLUSTER=1
Environment=WORKERS=4
EnvironmentFile=/srv/aibuild/backend/.env
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=5
StandardOutput=append:/var/log/tia-api.log
StandardError=append:/var/log/tia-api.err

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable --now tia-api
sudo systemctl status tia-api
```

7.3 Deploy the frontend

```
cd frontend
npm ci
npm run build              # -> dist/

# Serve dist/ from Vercel (recommended) or nginx/CDN.
# Leave VITE_API_URL unset when the API shares the domain (/api).
# Set VITE_SITE_URL only if the public origin differs from the served one.
```

7.4 Verify after deploying

```
curl https://your-domain.com/api/health
#   -> {"status":"ok","db":"3307"}

curl https://your-domain.com/api/categories | head -c 120
curl -I https://your-domain.com/logo.png          # press asset

# confirm the proxy header is being honoured: every request should show
# the real client IP, not the load balancer, in the votes table.

cd backend && node scripts/audit_endpoints.js     # 34 endpoint checks
```

7.5 Zero-downtime reload

```
sudo systemctl reload tia-api      # not needed with cluster master
# or, for a manual rolling restart:
#   start the new instance on port 3002, health check it, then drain 3001.

# Verify each new instance before shifting traffic:
curl -s localhost:3002/api/health
```

7.6 Rollback

- Keep the previous build in /srv/aibuild/releases/<timestamp>/
- Point the systemd unit at the previous release and restart
- Database changes are additive and idempotent, so schema.sql can be re-applied safely
- Never roll back by dropping tables - votes and nominations are not reproducible

7.7 Backups

```
# nightly logical backup
mysqldump --single-transaction --routines --triggers \
  -u root -p telugu_icon_awards | gzip > /backups/tia-$(date +%F).sql.gz

# retention: keep 14 days, offsite
find /backups -name "tia-*.sql.gz" -mtime +14 -delete
```

--single-transaction gives a consistent snapshot without locking tables. Test a restore before launch day - an untested backup is not a backup.

8. Pre-Launch Checklist

8.1 Must fix before going live

1. Buffer pool
innodb_buffer_pool_size is 16MB (XAMPP default). Raise to 1G or more. Past a few tens of thousands of votes the votes table stops fitting in memory and every write hits disk.

2. OTP verification
If MSG91_TEMPLATE_ID is empty, /api/otp/send returns the code in the response as dev_otp. Anyone can vote without owning the number. Either configure MSG91 or reject unverified votes server-side.

3. Default admin password
admin@teluguiconawards.com / admin123 is in schema.sql. Change it before the site is reachable.

4. TRUST_PROXY
Without it every visitor shares one IP, so 20 requests exhaust the hourly limit for everybody and the first person to vote locks out the entire audience.

8.2 Recommended before going live

5. Remove the vote_count write lock
The largest single throughput win available. Move the counter out of the vote transaction and derive totals from the votes table, or accumulate into a cache and flush periodically. This removes ~10,600 row-lock waits per 16,000 votes and the 0.07% deadlock failures.

- 6. Set DB_POOL_SIZE and max_connections consistently for your instance count
- 7. Enable slow-query logging before launch, review it after
- 8. Put the frontend behind a CDN so read traffic never reaches Express
- 9. Test the database restore procedure
- 10. Load test the real deployment at the expected peak, not just localhost

8.3 Launch-day monitoring

SYMPTOM	LIKELY CAUSE
`p99 > 1000 ms`	DB pool saturated, or lock contention. Check slow log.
`503 from /api/votes`	isBusyError fired - pool or queue pressure. Add instances.
`"Too many connections"`	max_connections < instances x DB_POOL_SIZE.
`0.07% HTTP 500 on votes`	Deadlock retries exhausted. See item 5 above.
`Duplicate votes reported`	TRUST_PROXY not set - all users share one IP.

8.4 Emergency levers

LEVER	EFFECT
`CLUSTER=0, WORKERS=1`	Restart. Frees memory if one instance is wedged.
`DB_POOL_SIZE down`	Relieves a saturated pool at the cost of throughput.
`Raise VOTE_RATE_MAX`	Lets legitimate voters through during an SMS outage.
`Set voting_open=0`	Per-category kill switch from Admin > Categories.

End of runbook. Capacity figures are specific to the hardware in section 4 - re-run the load test after any change to the vote write path, the database settings, or the instance count.